

direct access to Google AI

Platform: Google Gemini | Generated: 5/30/2026

Here is the breakdown of the most optimal ways to reverse a String in Java, both without and with the Stream API. Since strings in Java are immutable, any approach to reversing a string will require $O(n)$ space complexity to store the new character sequence.

1. Without Streams (Optimal Approaches)

When avoiding streams, you have two main options depending on the context of the question: the algorithmic approach (best for interviews to show you understand pointers) and the practical approach (best for production code).

A. The Algorithmic Approach: Two Pointers

This is the most optimal manual way to reverse a string. It converts the string to a character array and swaps characters from the outside in.

```

Java
public class StringReverser {
    public static String reverseWithPointers(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        char[] chars = str.toCharArray();
        int left = 0;
        int right = chars.length - 1;

        while (left < right) {
            // Swap the characters
            char temp = chars[left];
            chars[left] = chars[right];
            chars[right] = temp;

            // Move pointers towards the center
            left++;
            right--;
        }

        return new String(chars);
    }
}

```

Time Complexity: $O(n)$ where n is the length of the string. We iterate through half the array, resulting in $n/2$ operations, which drops the constant to $O(n)$.

Space Complexity: $O(n)$ to create the `char[]` array.

B. The Practical Approach: StringBuilder

If the interviewer doesn't explicitly forbid built-in methods, this is the absolute standard and most optimized way to reverse a string in Java production environments.

```

Java
public class StringReverser {
    public static String reverseWithStringBuilder(String str) {
        if (str == null) return null;
        return new StringBuilder(str).reverse().toString();
    }
}

```

Explanation: `StringBuilder` is backed by a mutable array. Its `.reverse()` method is heavily optimized at the JVM level (using intrinsic locks and SIMD instructions on modern hardware).

2. With Streams (Optimal Approach)

A quick caveat: Java Streams are not inherently designed for primitive character manipulation. A naive approach (like converting to a list of `Character` objects and using `Collections.reverse`) introduces heavy boxing and unboxing overhead, which ruins performance. To

make a Stream approach optimal, we must avoid object boxing by using an IntStream and collecting the characters into a mutable container (StringBuilder).The IntStream Approach

```
import java.util.stream.IntStream;
```

```
public class StringReverser {
    public static String reverseWithStream(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        return IntStream.range(0, str.length())
            // Map the index to the character at the mirrored position
            .map(i -> str.charAt(str.length() - 1 - i))
            // Collect directly into a StringBuilder to avoid object boxing
            .collect(
                StringBuilder::new,          // Supplier: creates the StringBuilder
                (sb, c) -> sb.append((char) c), // Accumulator: appends the raw char
                StringBuilder::append        // Combiner: merges StringBuilders (for parallel streams)
            )
            .toString();
    }
}
```

Explanation: `IntStream.range(0, str.length())`: We generate a stream of primitive integers representing the indices of the string. Using `IntStream` avoids boxing integers into `Integer` objects. `.map(...)`: We map each index to the character on the opposite end of the string. For example, index 0 becomes the last character, index 1 becomes the second to last, etc. `.collect(...)`: Instead of converting the stream into objects to use standard Collectors, we use the 3-argument `.collect()` method native to `IntStream`. It provisions a `StringBuilder`. It appends the primitive characters directly to it. This ensures high efficiency and memory safety while adhering to the functional, declarative style of Streams. Time Complexity: $O(n)$ Space Complexity: $O(n)$

Here are the optimal implementations for each of the four algorithms. As you saw with the StringReverser, using Java Streams for primitive data types (like char or int) requires specific handling (like IntStream) to avoid heavy performance penalties from object boxing. Here are the complete classes, packed with optimal stream/non-stream methods, explanations, and edge-case testing in their respective run() methods. 1. PalindromeChecker A palindrome reads the same forwards and backwards. The optimal approach uses two pointers moving from the outside in, stopping at the middle. package.com/codes/java/codingquestions;

```
import java.util.stream.IntStream;
```

```
public class PalindromeChecker {
```

```
// 1. Without Streams (Optimal Two-Pointer: O(n) Time, O(1) Space)
```

```
public static boolean isPalindrome(String str) {
```

```
    if (str == null) return false;
```

```
    if (str.length() <= 1) return true;
```

```
    int left = 0;
```

```
    int right = str.length() - 1;
```

```
    while (left < right) {
```

```
        if (str.charAt(left) != str.charAt(right)) {
```

```
            return false;
```

```
        }
```

```
        left++;
```

```
        right--;
```

```
    }
```

```
    return true;
```

```
}
```

```
// 2. With Streams (Optimal IntStream: O(n) Time, O(1) Space)
```

```
public static boolean isPalindromeStream(String str) {
```

```
    if (str == null) return false;
```

```
    int len = str.length();
```

```
    if (len <= 1) return true;
```

```
    // Only check up to the middle of the string
```

```
    return IntStream.range(0, len / 2)
```

```
        .noneMatch(i -> str.charAt(i) != str.charAt(len - i - 1));
```

```
}
```

```
// Test Cases
```

```
public static void run() {
```

```
    System.out.println("--- Palindrome Checker ---");
```

```
    String[] tests = {"racecar", "Lawrence", "a", "", null};
```

```

    for (String test : tests) {
        System.out.println("Input: " + test +
            " | Pointer: " + isPalindrome(test) +
            " | Stream: " + isPalindromeStream(test));
    }
}

```

Explanation: Both approaches run in $O(n)$ time. The stream approach uses `IntStream.range` to iterate only up to halfway ($\text{len} / 2$). The `.noneMatch()` operator guarantees an early exit (short-circuiting) the moment a mismatch is found, mimicking the efficiency of the while loop.
 2. AnagramCheckerTwo strings are anagrams if they contain the exact same characters with the same frequencies.
 Javapackage
 com.codes.java.codingquestions;

```
import java.util.Arrays;
```

```
public class AnagramChecker {
```

```
    // 1. Without Streams (Optimal Frequency Array:  $O(n)$  Time,  $O(1)$  Space)
```

```
    public static boolean isAnagram(String s1, String s2) {
        if (s1 == null || s2 == null || s1.length() != s2.length()) {
            return false;
        }
    }

```

```
    // Assuming ASCII characters. Use HashMap if Unicode is required.
```

```
    int[] charCounts = new int[256];
```

```
    for (int i = 0; i < s1.length(); i++) {
        charCounts[s1.charAt(i)]++;
        charCounts[s2.charAt(i)]--;
    }

```

```
    // If it's an anagram, all counts will have balanced back to 0
```

```
    for (int count : charCounts) {
        if (count != 0) return false;
    }
    return true;
}

```

```
    // 2. With Streams (Sorting approach:  $O(n \log n)$  Time,  $O(n)$  Space)
```

```
    public static boolean isAnagramStream(String s1, String s2) {
        if (s1 == null || s2 == null || s1.length() != s2.length()) {
            return false;
        }
    }

```

```
    // Sorting is the most idiomatic stream way, though slightly slower than frequency array
```

```
    int[] sorted1 = s1.chars().sorted().toArray();
    int[] sorted2 = s2.chars().sorted().toArray();

```

```

        return Arrays.equals(sorted1, sorted2);
    }

    // Test Cases
    public static void run() {
        System.out.println("\n--- Anagram Checker ---");
        String[][] tests = {
            {"listen", "silent"},
            {"hello", "world"},
            {"a", "a"},
            {"triangle", "integral"},
            {null, "test"}
        };

        for (String[] test : tests) {
            System.out.println("Input: [" + test[0] + ", " + test[1] + "]" +
                " | Array: " + isAnagram(test[0], test[1]) +
                " | Stream: " + isAnagramStream(test[0], test[1]));
        }
    }
}

```

Explanation: The non-stream version is heavily optimized: it uses a single array of size 256 to track character occurrences, adding for string 1 and subtracting for string 2. The Stream version uses `.chars().sorted()`. While Streams can do frequency counting (via `Collectors.groupingBy`), it forces primitive char values to box into Character objects, causing massive memory overhead. Sorting is the cleaner, more practical Stream solution here.

3. FibonacciSeries

Generating the Fibonacci series up to n terms. We return `long[]` because Fibonacci numbers grow exponentially and quickly exceed standard 32-bit integer limits.

Java package `com.codes.java.codingquestions`;

```

import java.util.Arrays;
import java.util.stream.Stream;

public class FibonacciSeries {

    // 1. Without Streams (Optimal Iterative: O(n) Time, O(n) Space)
    public static long[] generate(int n) {
        if (n <= 0) return new long[0];
        if (n == 1) return new long[]{0};

        long[] fib = new long[n];
        fib[0] = 0;
        fib[1] = 1;

        for (int i = 2; i < n; i++) {
            fib[i] = fib[i - 1] + fib[i - 2];
        }
        return fib;
    }
}

```

```
// 2. With Streams (Iterate approach: O(n) Time, O(n) Space)
```

```
public static long[] generateStream(int n) {  
    if (n <= 0) return new long[0];  
  
    return Stream.iterate(new long[]{0, 1}, t -> new long[]{t[1], t[0] + t[1]})  
        .limit(n)  
        .mapToLong(t -> t[0])  
        .toArray();  
}
```

```
// Test Cases
```

```
public static void run() {  
    System.out.println("\n--- Fibonacci Series ---");  
    int[] tests = {0, 1, 5, 10};  
  
    for (int n : tests) {  
        System.out.println("n=" + n + " | Loop: " + Arrays.toString(generate(n)));  
        System.out.println("n=" + n + " | Stream: " + Arrays.toString(generateStream(n)));  
    }  
}
```

Explanation: The traditional loop simply populates an array. The Stream approach leverages `Stream.iterate`, holding the "current" and "next" values in a 2-element array `new long[]{t[1], t[0] + t[1]}`. We map to extract just the first element `t[0]` for our final array. This is mathematically elegant and fully immutable.

4. PrimeNumberA prime number is a number greater than 1 that has no positive divisors other than 1 and itself.
javapackage.com/codes/java/codingquestions/

```
import java.util.stream.IntStream;
```

```
public class PrimeNumber {
```

```
// 1. Without Streams (Optimal Math: O(√n) Time, O(1) Space)
```

```
public static boolean isPrime(int n) {  
    if (n <= 1) return false;  
    if (n == 2 || n == 3) return true;  
    if (n % 2 == 0) return false; // fast check for even numbers  
  
    // Check odd numbers up to the square root of n  
    for (int i = 3; i <= Math.sqrt(n); i += 2) {  
        if (n % i == 0) return false;  
    }  
    return true;  
}
```

```
// 2. With Streams (Optimal IntStream: O(√n) Time, O(1) Space)
```

```
public static boolean isPrimeStream(int n) {  
    if (n <= 1) return false;
```

```

    if (n == 2) return true;

    // rangeClosed creates a stream from 2 to √n
    return IntStream.rangeClosed(2, (int) Math.sqrt(n))
        .noneMatch(i -> n % i == 0);
}

// Test Cases
public static void run() {
    System.out.println("\n--- Prime Number Checker ---");
    int[] tests = {-5, 0, 1, 2, 3, 4, 17, 100};

    for (int test : tests) {
        System.out.println("Input: " + test +
            " | Loop: " + isPrime(test) +
            " | Stream: " + isPrimeStream(test));
    }
}

```

Explanation: Both algorithms utilize the mathematical rule that a number's largest unique factor cannot exceed its square root, meaning time complexity drops drastically from $O(n)$ to $O(\sqrt{n})$. The Stream method utilizes `.noneMatch()` which guarantees short-circuiting—the stream stops executing the instant it finds a divisor, making it exactly as performant as the for loop break mechanism.

Google Gemini

A palindrome is a word, phrase, number, or any sequence of characters that reads the exact same forward and backward. When looking at palindromes, we generally ignore spaces, punctuation, and capitalization to see the core pattern. Here are a few classic examples broken down by category:

- Palindrome Words** These are single words that mirror perfectly from the center. `racecar`, `level`, `radar`, `civic`, `kayak`.
- Palindrome Phrases** When you remove the spaces and punctuation, the sequence of letters is symmetrical. "A man, a plan, a canal: Panama!" (Reads: `amanaplanacanalpanama`) "Was it a car or a cat I saw?" (Reads: `wasitacaroracatisaw`) "Never odd or even" (Reads: `neveroddoeven`) "Taco cat" (Reads: `tacocat`).
- Palindrome Numbers** Numbers that read the same both ways. These frequently pop up in math puzzles. `121`, `1331`, `2020`.

In the context of the Java code we just went through, the `PalindromeChecker` methods I provided were designed for single words without spaces. If you pass "Taco cat" into our previous method, it would return false because the space is at index 4 going forward, but index 3 going backward! Would you like to see how to update our Java code to correctly handle full phrases by filtering out spaces and punctuation?

Finding the First Non-Repeating Character in a string is a classic interview question. The goal is to traverse the string and return the first character that only appears exactly once. Here is the complete class, featuring both the traditional highly-optimized approach and the modern Stream API approach. `FirstNonRepeatingCharacter.java` package `com.codes.java.codingquestions`;

```
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

public class FirstNonRepeatingCharacter {

    // 1. Without Streams (Optimal Array: O(n) Time, O(1) Space)
    public static Character findFirst(String str) {
        if (str == null || str.isEmpty()) {
            return null;
        }

        // Assuming standard ASCII characters.
        int[] charCounts = new int[256];

        // First pass: Count the frequency of each character
        for (int i = 0; i < str.length(); i++) {
            charCounts[str.charAt(i)]++;
        }

        // Second pass: Find the first character with a frequency of exactly 1
        for (int i = 0; i < str.length(); i++) {
            if (charCounts[str.charAt(i)] == 1) {
                return str.charAt(i);
            }
        }

        return null; // Return null if all characters repeat
    }

    // 2. With Streams (LinkedHashMap: O(n) Time, O(n) Space)
    public static Character findFirstStream(String str) {
        if (str == null || str.isEmpty()) {
            return null;
        }

        return str.chars()           // Create an IntStream of characters
            .mapToObj(c -> (char) c) // Box primitive ints to Character objects
            .collect(Collectors.groupingBy(
                Function.identity(), // The character itself is the key
```

```

        LinkedHashMap::new,    // CRITICAL: LinkedHashMap preserves insertion order
        Collectors.counting() // Count occurrences
    ))
    .entrySet().stream()      // Stream the map entries
    .filter(e -> e.getValue() == 1L) // Filter out repeating characters
    .map(Map.Entry::getKey)   // Extract the character
    .findFirst()              // Get the first one that matches
    .orElse(null);
}

// Test Cases
public static void run() {
    System.out.println("--- First Non-Repeating Character ---");
    String[] tests = {
        "swiss",    // 'w'
        "programming", // 'p'
        "aabbcc",   // null
        "Lawrence", // 'L'
        null,
        ""
    };

    for (String test : tests) {
        System.out.println("Input: " + (test == null ? "null" : "\"" + test + "\"") +
            " | Array: " + findFirst(test) +
            " | Stream: " + findFirstStream(test));
    }
}

```

Explanation of the Approaches

- Without Streams (The Frequency Array)** This is the most optimal approach for performance. It uses a time complexity of $O(n)$ and a constant space complexity of $O(1)$ (since the array size is fixed at 256 regardless of the string's length). It requires two passes: the first pass increments a counter in an array based on the character's ASCII value. The second pass walks through the string again from left to right, looking up the count in the array. The very first character it finds with a count of 1 is returned, guaranteeing we get the first one in the string.
- With Streams (The LinkedHashMap Approach)** This is the idiomatic functional approach, but it uses $O(n)$ space complexity due to object boxing and map creation. Standard HashMaps do not guarantee order. Because we need the first character, we must instruct the `Collectors.groupingBy` method to explicitly use a `LinkedHashMap`, which maintains the order in which the characters were originally inserted. Once the map is built with the counts, we stream the map entries, filter for counts exactly equal to 1L, and grab the first match.

Counting character frequency is another standard interview question that heavily builds on the concepts we used in the "First Non-Repeating Character" problem. While you could use a flat integer array (int[256]) for peak ASCII performance, returning a Map is the industry standard for this question because it handles Unicode seamlessly and scales to any character set without allocating unused memory. Here is the complete class, featuring both approaches. `CharacterFrequency` package `com.codes.java.codingquestions`;

```
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

public class CharacterFrequency {

    // 1. Without Streams (Optimal HashMap: O(n) Time, O(k) Space)
    public static Map<Character, Integer> count(String str) {
        if (str == null || str.isEmpty()) {
            return Collections.emptyMap(); // Safe, immutable empty map
        }

        // Using LinkedHashMap to maintain the insertion order of characters
        Map<Character, Integer> frequencyMap = new LinkedHashMap<>();

        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            // getOrDefault handles the null check if the key doesn't exist yet
            frequencyMap.put(c, frequencyMap.getOrDefault(c, 0) + 1);
        }

        return frequencyMap;
    }

    // 2. With Streams (Optimal Stream: O(n) Time, O(k) Space)
    public static Map<Character, Long> countStream(String str) {
        if (str == null || str.isEmpty()) {
            return Collections.emptyMap();
        }

        return str.chars()           // 1. Create an IntStream of characters
            .mapToObj(c -> (char) c) // 2. Box primitive ints into Character objects
            .collect(Collectors.groupingBy( // 3. Collect and group them
                Function.identity(), // The key is the character itself
                LinkedHashMap::new, // Force a LinkedHashMap to preserve order
                Collectors.counting() // The value is the count of occurrences
            ));
    }
}
```

```

}

// Test Cases
public static void run() {
    System.out.println("--- Character Frequency ---");
    String[] tests = {
        "hello",
        "programming",
        "Lawrence",
        "",
        null
    };

    for (String test : tests) {
        String displayStr = (test == null) ? "null" : "\"" + test + "\"";
        System.out.println("Input: " + displayStr);
        System.out.println(" Loop: " + count(test));
        System.out.println(" Stream: " + countStream(test));
        System.out.println();
    }
}
}

```

Explanation of the Approaches

Note on Space Complexity: Both approaches have a space complexity of $O(k)$, where k is the number of unique characters in the string.

- Without Streams (The `getOrDefault` Approach)** This is the cleanest and most efficient way to write this iteratively in modern Java. We use a `LinkedHashMap`. Unlike a standard `HashMap`, this preserves the order in which characters first appear in the string, which makes the output predictable and much easier to read during debugging. The magic happens with `frequencyMap.getOrDefault(c, 0) + 1`. In older versions of Java, you had to write an `if/else` block to check if the map already contained the key. `getOrDefault` fetches the current count, but if the character hasn't been seen yet, it defaults to 0, adds 1 to it, and puts it in the map.
- With Streams (The `Collectors.groupingBy` Approach)** This is the standard declarative approach using Java Streams. `str.chars()` gets us a stream of primitive integers (representing the characters). We box them into `Character` objects using `.mapToObj()`. We use `Collectors.groupingBy` to build our Map. `Function.identity()` is just a fancy, functional way of saying "use the item itself as the key." `Collectors.counting()` is a built-in downstream collector that simply tallies up how many times it sees each key. It natively returns a `Long`, which is why the return type of the stream method is `Map<Character, Long>` instead of `Integer`.

While the previous question asked for the frequencies of all characters in a string, this specific question asks you to count how many times a single target character appears. Because we only need to track one character instead of building a whole Map, the space complexity drops to $O(1)$. Here is the complete class, featuring the traditional loop and the functional stream approaches. [CharacterOccurrenceCounter.javapackage com.codes.java.codingquestions;](https://www.javapackages.com/codes/java/codingquestions/)

```
public class CharacterOccurrenceCounter {

    // 1. Without Streams (Optimal Loop: O(n) Time, O(1) Space)
    public static int countOccurrence(String str, char target) {
        if (str == null || str.isEmpty()) {
            return 0;
        }

        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) == target) {
                count++;
            }
        }

        return count;
    }

    // 2. With Streams (Optimal Stream: O(n) Time, O(1) Space)
    public static long countOccurrenceStream(String str, char target) {
        if (str == null || str.isEmpty()) {
            return 0;
        }

        return str.chars()           // 1. Create an IntStream of characters
            .filter(c -> c == target) // 2. Filter only for the target character
            .count();                 // 3. Count the remaining elements
    }

    // Test Cases
    public static void run() {
        System.out.println("--- Count Occurrence of Specific Character ---");

        String testString = "programming in java";
        char[] targets = {'r', 'g', 'a', 'z'};

        System.out.println("Input String: \"" + testString + "\"");

        for (char target : targets) {
            System.out.println("Target: " + target + " " +
```

```

        " | Loop: " + countOccurrence(testString, target) +
        " | Stream: " + countOccurrenceStream(testString, target));
    }

    // Edge Cases
    System.out.println("Target: 'a' | Loop (null): " + countOccurrence(null, 'a') +
        " | Stream (null): " + countOccurrenceStream(null, 'a'));
}
}

```

Explanation of the Approaches

1. Without Streams (The for Loop) This is the most straightforward and performant way to solve the problem. We initialize a simple integer count. We iterate over the string exactly once (Time Complexity: $O(n)$) using `.charAt(i)`. Every time we find a match, we increment the counter. It allocates zero extra memory structure (Space Complexity: $O(1)$).
2. With Streams (The `filter().count()` Approach) This is a perfectly optimized Stream approach because it entirely avoids object boxing (turning primitive `char/int` into `Character/Integer` objects). `str.chars()` creates an `IntStream` (primitives). `.filter(c -> c == target)` keeps the stream in primitive space, only passing along characters that match our target. `.count()` is a built-in terminal operation that tallies the surviving elements. Note that Stream's `.count()` always returns a `long` in Java, which is why our method signature returns `long` instead of `int`.

Here are the optimal implementations for finding and removing duplicate characters in a string, following the same structure and optimization standards used previously.

1. DuplicateCharacters (Finding Duplicates)

The goal of this question is to identify and return all characters that appear more than once in the input string.

Java package: `com.codes.java.codingquestions`;

```
import java.util.Collections;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Map;
import java.util.Set;
import java.util.function.Function;
import java.util.stream.Collectors;

public class DuplicateCharacters {

    // 1. Without Streams (Optimal Two-Set Approach: O(n) Time, O(k) Space)
    public static Set<Character> findDuplicates(String str) {
        if (str == null || str.isEmpty()) {
            return Collections.emptySet();
        }

        Set<Character> seen = new HashSet<>();
        // LinkedHashSet keeps the order in which duplicates are discovered
        Set<Character> duplicates = new LinkedHashSet<>();

        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            // If the character cannot be added to 'seen', it means it's a duplicate
            if (!seen.add(c)) {
                duplicates.add(c);
            }
        }
        return duplicates;
    }

    // 2. With Streams (Optimal Grouping Approach: O(n) Time, O(k) Space)
    public static Set<Character> findDuplicatesStream(String str) {
        if (str == null || str.isEmpty()) {
            return Collections.emptySet();
        }

        return str.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
            .entrySet().stream()
            .filter(entry -> entry.getValue() > 1) // Filter out unique characters
    }
}
```

```

        .map(Map.Entry::getKey)
        .collect(Collectors.toCollection(LinkedHashSet::new));
    }

    // Test Cases
    public static void run() {
        System.out.println("--- Find Duplicate Characters ---");
        String[] tests = {"programming", "Java", "abcdef", "", null};

        for (String test : tests) {
            String displayStr = (test == null) ? "null" : "\"" + test + "\"";
            System.out.println("Input: " + displayStr +
                " | Loop: " + findDuplicates(test) +
                " | Stream: " + findDuplicatesStream(test));
        }
    }
}

```

Explanation of the Approaches Without Streams: We iterate through the string once. `Set.add()` returns false if the item is already present in the set. We leverage this property to immediately capture duplicates into a separate `LinkedHashSet` (ensuring the output matches the order of discovery). This avoids doing a secondary pass or allocating massive frequency tables. With Streams: We use `groupingBy` to map out the frequency of every single character. From there, we convert the map back into a stream of entries, throw away anything with a count of 1, and harvest the keys into an ordered set.

2. RemoveDuplicateCharacters (Keeping Only Unique/First Occurrences) The goal here is to strip out subsequent occurrences of a character, keeping only its very first appearance, preserving the original string's relative order.

javapackage.com/codes/java/codingquestions/

```

import java.util.stream.Collectors;

public class RemoveDuplicateCharacters {

    // 1. Without Streams (Optimal Array Tracker: O(n) Time, O(1) Space)
    public static String removeDuplicates(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        // Assuming ASCII characters. Use a Set<Character> if Unicode is required.
        boolean[] seen = new boolean[256];
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            if (!seen[c]) {
                seen[c] = true; // Mark as seen
                sb.append(c);    // Append to our unique result sequence
            }
        }
    }
}

```



```

        return sb.toString();
    }

    // 2. With Streams (Highly Optimized Primitive Stream: O(n) Time, O(k) Space)
    public static String removeDuplicatesStream(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        return str.chars()           // Returns an IntStream of code points (primitive values)
            .distinct()              // Standard intermediate stream method to eliminate duplicates
            // Collect directly into a StringBuilder using primitive appendCodePoint to avoid boxing
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();
    }

    // Test Cases
    public static void run() {
        System.out.println("\n--- Remove Duplicate Characters ---");
        String[] tests = {"programming", "bananas", "aabbcc", "", null};

        for (String test : tests) {
            String displayStr = (test == null) ? "null" : "\"" + test + "\"";
            System.out.println("Input: " + displayStr);
            System.out.println(" Loop:  \"\" + removeDuplicates(test) + \"\"");
            System.out.println(" Stream: \"\" + removeDuplicatesStream(test) + \"\"");
        }
    }
}

```

Explanation of the Approaches Without Streams: For pure performance, a fixed-size boolean[256] lookup array acts as a toggle board. As we walk the string, we check if the flag at index c is true. If it isn't, we flip it to true and append to our string builder. This ensures constant space O(1) and linear time O(n). With Streams: The Stream API provides a native .distinct() operation. To build it optimally, we avoid converting the stream to Character objects. Instead, we pipe the raw primitive integers (IntStream) through .distinct() and collect them using StringBuilder::appendCodePoint. This retains pure execution speed in primitive memory space, making it highly optimal.

String Compression is a fantastic interview question because it tests your ability to handle sequential state (keeping track of previous characters) and introduces a classic "gotcha" regarding the length of the final string. A standard rule for String Compression (often popularized by Cracking the Coding Interview) is that if the compressed string is not strictly shorter than the original string, you should return the original string. For example, compressing "abc" into "a1b1c1" actually takes up more space, so we just return "abc". Here is a crucial detail regarding Java Streams: Streams are fundamentally designed to be stateless. They process elements independently (which makes them great for parallel processing). Because String Compression requires looking backward to see if the current character matches the previous one, pure Streams are the wrong tool for the job. Trying to force this into a Stream requires mutating external state or writing a massive, ugly custom Collector. Instead, for your second option, I have provided the Declarative Regex Approach. This shows the interviewer you understand how to write functional, declarative code without forcing Streams where they don't belong.

StringCompressionJavapackage com.codes.java.codingquestions;

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
```

```
public class StringCompression {
```

```
    // 1. Standard Approach (Optimal Iteration: O(n) Time, O(n) Space)
```

```
    public static String compress(String str) {
```

```
        if (str == null || str.isEmpty()) {
```

```
            return str;
```

```
        }
```

```
        StringBuilder compressed = new StringBuilder();
```

```
        int countConsecutive = 0;
```

```
        for (int i = 0; i < str.length(); i++) {
```

```
            countConsecutive++;
```

```
            // If we are at the last character OR the next character is different
```

```
            if (i + 1 >= str.length() || str.charAt(i) != str.charAt(i + 1)) {
```

```
                compressed.append(str.charAt(i));
```

```
                compressed.append(countConsecutive);
```

```
                // Reset count for the next sequence of characters
```

```
                countConsecutive = 0;
```

```
            }
```

```
        }
```

```
        // Only return the compressed string if it actually saves space
```

```
        return compressed.length() < str.length() ? compressed.toString() : str;
```

```
    }
```

```
    // 2. Declarative Approach (Using Regex Matcher: O(n) Time, O(n) Space)
```

```

public static String compressDeclarative(String str) {
    if (str == null || str.isEmpty()) {
        return str;
    }

    StringBuilder compressed = new StringBuilder();

    // Match any character "(.)" and then match that exact same character zero or more times "\\1*"
    Matcher matcher = Pattern.compile("(.)\\1*").matcher(str);

    while (matcher.find()) {
        // matcher.group(1) is the single character. matcher.group() is the whole repeated sequence.
        compressed.append(matcher.group(1));
        compressed.append(matcher.group().length());
    }

    return compressed.length() < str.length() ? compressed.toString() : str;
}

// Test Cases
public static void run() {
    System.out.println("--- String Compression ---");
    String[] tests = {
        "aabccccaaa", // Expected: a2b1c5a3
        "abcd",       // Expected: abcd (a1b1c1d1 is longer)
        "nnn",        // Expected: n3
        "a",           // Expected: a
        "aabbcc",     // Expected: aabbcc (a2b2c2 is same length, return original)
        "",
        null
    };

    for (String test : tests) {
        String displayStr = (test == null) ? "null" : "\"" + test + "\"";
        System.out.println("Input: " + displayStr);
        System.out.println(" Loop: " + compress(test));
        System.out.println(" Regex: " + compressDeclarative(test));
        System.out.println();
    }
}

```

Explanation of the Approaches

- 1. The Standard Approach (The for Loop)** This is exactly what an interviewer wants to see.
 - Time Complexity:** $O(n)$ because we iterate through the characters exactly once.
 - Space Complexity:** $O(n)$ because we allocate a `StringBuilder` that scales linearly with the input.
 - The Logic:** We increment a counter on every loop. We use a peek-ahead technique `str.charAt(i) != str.charAt(i + 1)` to detect boundaries. If the boundary changes, we flush our character and counter to the `StringBuilder` and reset the counter to zero.
- 2. The Declarative Approach (Regex)** If an interviewer asks, "How would you do this functionally or concisely without writing manual iteration logic?" this is

your ace in the hole. The Pattern: `"(.\\1*)"` is a brilliant regular expression for this problem. `(.)` captures any single character into "Group 1". `\\1*` is a backreference. It tells the engine to look at what it just captured in Group 1, and match it zero or more times consecutively. This groups identical consecutive characters together automatically. `m.group(1)` gives us the character, and `m.group().length()` gives us the total count of how many times it repeated in that block.

Finding the Longest Common Prefix (LCP) is a highly popular array and string manipulation question. The goal is to find the longest initial sequence of characters that all strings in an array share. If there is no common prefix, you return an empty string "". Here is the complete class, featuring the classic optimal approach and the modern Stream API approach using reduction. [LongestCommonPrefixJava package com.codes.java.codingquestions;](https://www.comcodes.com/java/codingquestions/)

```
import java.util.Arrays;
```

```
public class LongestCommonPrefix {
```

```
    // 1. Without Streams (Optimal Horizontal Scanning: O(S) Time, O(1) Space)
```

```
    public static String findPrefix(String[] strs) {
```

```
        if (strs == null || strs.length == 0) {
```

```
            return "";
```

```
        }
```

```
        // Start by assuming the first string is the common prefix
```

```
        String prefix = strs[0];
```

```
        for (int i = 1; i < strs.length; i++) {
```

```
            // While the current string doesn't start with the prefix...
```

```
            while (!strs[i].startsWith(prefix)) {
```

```
                // Chop off the last character of the prefix and try again
```

```
                prefix = prefix.substring(0, prefix.length() - 1);
```

```
            // Early exit: if the prefix shrinks to nothing, there is no common prefix
```

```
            if (prefix.isEmpty()) {
```

```
                return "";
```

```
            }
```

```
        }
```

```
    }
```

```
    return prefix;
```

```
}
```

```
    // 2. With Streams (Reduction Approach: O(S) Time, O(1) Space)
```

```
    public static String findPrefixStream(String[] strs) {
```

```
        if (strs == null || strs.length == 0) {
```

```
            return "";
```

```
        }
```

```
        return Arrays.stream(strs)
```

```
            // reduce() continually takes the accumulated prefix and the next string
```

```
            .reduce((prefix, str) -> {
```

```
                while (!str.startsWith(prefix)) {
```

```
                    prefix = prefix.substring(0, prefix.length() - 1);
```

```

        }
        return prefix;
    })
    .orElse("");
}

// Test Cases
public static void run() {
    System.out.println("--- Longest Common Prefix ---");

    String[][] tests = {
        {"flower", "flow", "flight"},           // Expected: "fl"
        {"interspecies", "interstellar", "interstate"}, // Expected: "inters"
        {"dog", "racecar", "car"},               // Expected: "" (No common prefix)
        {"throne", "throne"},                   // Expected: "throne"
        {"", ""},                               // Expected: ""
        null                                     // Expected: ""
    };

    for (String[] test : tests) {
        String displayArr = (test == null) ? "null" : Arrays.toString(test);
        System.out.println("Input: " + displayArr);
        System.out.println(" Loop: \\" + findPrefix(test) + "\\"");
        System.out.println(" Stream: \\" + findPrefixStream(test) + "\\"");
        System.out.println();
    }
}

```

Explanation of the Approaches
Note on Time Complexity: Both approaches operate in $O(S)$ time, where S is the sum of all characters in all strings in the array. In the worst-case scenario (where all strings are identical), every character is evaluated.

- Without Streams (Horizontal Scanning)** This is the standard, most optimal algorithmic approach. We initialize the prefix variable using the entirety of the first string (e.g., "flower"). We loop through the remaining strings. For the next string ("flow"), we check if it starts with "flower". It doesn't, so we chop off the last letter of our prefix (becoming "flowe"). We repeat this chopping until "flow".startsWith("flow") returns true. We then move to the next string ("flight") and repeat the process, shrinking "flow" down to "fl". The `if (prefix.isEmpty())` check acts as a highly efficient short-circuit—if our prefix ever drops to an empty string, we stop the whole loop immediately because it's impossible to find a longer match later.
- With Streams (`Arrays.stream().reduce()`)** Streams are generally designed to process elements independently, but this problem requires keeping track of an accumulating state (the shrinking prefix). We use the reduce terminal operation, which is perfect for this. It takes an accumulator (our prefix) and the next element in the stream (the str). Inside the lambda, we apply the exact same startsWith shrinking logic. The returned prefix from the lambda becomes the accumulator for the next string in the pipeline. `orElse("")` handles the edge case where the array is empty, safely returning a blank string.

Removing whitespaces from a string is a very common task. In an interview setting, the key difference between an amateur and an expert is knowing that a space (" ") is not the only kind of whitespace. A robust solution must also handle tabs (\t), newlines (\n), and carriage returns (\r). Fortunately, Java provides the built-in `Character.isWhitespace()` method to handle this perfectly. Here is the complete class featuring the manual iterative approach, the standard Stream approach, and the built-in Regex approach used in production.

```

package com.codes.java.codingquestions;

public class WhitespaceRemover {

    // 1. Without Streams (Optimal Iteration: O(n) Time, O(n) Space)
    public static String remove(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        // Initialize StringBuilder with original length to prevent array resizing overhead
        StringBuilder sb = new StringBuilder(str.length());

        for (int i = 0; i < str.length(); i++) {
            char c = str.charAt(i);
            // Character.isWhitespace catches spaces, tabs, newlines, etc.
            if (!Character.isWhitespace(c)) {
                sb.append(c);
            }
        }

        return sb.toString();
    }

    // 2. With Streams (Primitive Stream: O(n) Time, O(n) Space)
    public static String removeStream(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        return str.chars() // Generate IntStream of characters
            .filter(c -> !Character.isWhitespace(c)) // Keep only non-whitespaces
            // Collect directly into StringBuilder using primitive appendCodePoint
            .collect(StringBuilder::new, StringBuilder::appendCodePoint, StringBuilder::append)
            .toString();
    }

    // 3. The Production Approach (Regex)
    public static String removeRegex(String str) {
        if (str == null) return null;
    }

```

```

// \s+ matches one or more whitespace characters and replaces them with nothing
return str.replaceAll("\s+", "");
}

// Test Cases
public static void run() {
    System.out.println("--- Remove Whitespaces ---");

    String[] tests = {
        "hello world",          // Standard space
        " java  programming ", // Multiple spaces and leading/trailing
        "tab\tseparated\twords", // Tabs
        "multi\nline\rstring",  // Newlines and carriage returns
        "nospaces",             // No changes needed
        "",
        null
    };

    for (String test : tests) {
        // Format the output string so tabs/newlines are visible in the console
        String displayStr = (test == null) ? "null"
            : "\"" + test.replace("\n", "\\n").replace("\r", "\\r").replace("\t", "\\t") + "\"";

        System.out.println("Input: " + displayStr);
        System.out.println(" Loop:  \"" + remove(test) + "\"");
        System.out.println(" Stream:  \"" + removeStream(test) + "\"");
        System.out.println(" Regex:  \"" + removeRegex(test) + "\"");
        System.out.println();
    }
}
}

```

Explanation of the Approaches

Note on Space Complexity: Since Java Strings are immutable, any approach that modifies a string requires $O(n)$ space complexity to build the new resulting string.

- Without Streams (The for Loop)** This is the most highly optimized manual approach. We initialize the `StringBuilder` with the exact length of the original string (`str.length()`). Even though the final string will likely be shorter, pre-sizing the `StringBuilder` guarantees that its underlying array will never have to resize itself during the loop, shaving off microscopic execution time. We loop through the characters and only append them if they fail the `Character.isWhitespace()` check.
- With Streams (`IntStream` Filtering)** This maintains optimal speed while utilizing functional syntax. We use `str.chars()` to create a primitive stream, avoiding object boxing. We use `.filter()` to strip out the whitespaces. We collect the surviving characters straight into a `StringBuilder` using `appendCodePoint`, which safely appends raw integer values as standard characters without converting them into `Character` objects first.
- The Production Approach (Regex)** If you are doing this at your day job rather than in a coding interview, you should just use `str.replaceAll("\s+", "")`. `\s` is the regular expression for whitespace. `+` means "one or more consecutive whitespaces". It relies on Java's highly optimized Regex engine to strip out all spaces, tabs, and newlines in a single, easily readable line of code.

Checking if one string is a rotation of another is a classic trick question (famously popularized by Cracking the Coding Interview). A string is a rotation if you can cut it at some index and swap the two halves to form the other string (e.g., "waterbottle" is a rotation of "erbottlewat"). This is a scenario where Streams are not the optimal approach, but they can be used to write a purely functional offset-checker. However, the standard "concatenation trick" is what every interviewer is secretly waiting for you to use. Here is the complete class featuring both approaches. RotatedStringChecker package com.codes.java.codingquestions;

```
import java.util.stream.IntStream;
```

```
public class RotatedStringChecker {
```

```
    // 1. Without Streams (The Optimal Concatenation Trick: O(n) Time, O(n) Space)
```

```
    public static boolean isRotation(String s1, String s2) {
```

```
        // Step 1: Check for nulls and ensure lengths match
```

```
        if (s1 == null || s2 == null || s1.length() != s2.length()) {
```

```
            return false;
```

```
        }
```

```
        // Step 2: Handle empty strings
```

```
        if (s1.isEmpty()) {
```

```
            return true;
```

```
        }
```

```
        // Step 3: The magic trick
```

```
        String concatenated = s1 + s1;
```

```
        // If s2 is a rotation, it MUST be a substring of s1 concatenated with itself
```

```
        return concatenated.contains(s2);
```

```
    }
```

```
    // 2. With Streams (Declarative Offset Matching: O(n^2) Time, O(1) Space)
```

```
    public static boolean isRotationStream(String s1, String s2) {
```

```
        if (s1 == null || s2 == null || s1.length() != s2.length()) {
```

```
            return false;
```

```
        }
```

```
        int n = s1.length();
```

```
        if (n == 0) return true;
```

```
        // Try every possible rotation offset 'i'
```

```
        return IntStream.range(0, n)
```

```
            .anyMatch(i ->
```

```
                // Check if all characters match at this offset
```

```
                IntStream.range(0, n)
```

```
                    .allMatch(j -> s1.charAt((i + j) % n) == s2.charAt(j))
```

```

    );
}

// Test Cases
public static void run() {
    System.out.println("--- Check Rotated Strings ---");

    String[][] tests = {
        {"waterbottle", "erbottlewat"}, // True
        {"hello", "llohe"},             // True
        {"java", "avaj"},                // False (This is reversed, not rotated)
        {"abc", "ab"},                   // False (Different lengths)
        {"", ""},                        // True
        {null, "test"}                   // False
    };

    for (String[] test : tests) {
        String s1 = (test[0] == null) ? "null" : "\"" + test[0] + "\"";
        String s2 = (test[1] == null) ? "null" : "\"" + test[1] + "\"";

        System.out.println("Input: s1=" + s1 + ", s2=" + s2);
        System.out.println(" Trick: " + isRotation(test[0], test[1]));
        System.out.println(" Stream: " + isRotationStream(test[0], test[1]));
        System.out.println();
    }
}

```

Explanation of the Approaches

- 1. The Standard Approach (The Concatenation Trick)** This is the undisputed most optimal and practical way to solve this problem. The Logic: If you split a string $s1$ into two parts, X and Y , such that $s1 = XY$, then a rotation of that string $s2$ will always be YX . If you concatenate $s1$ with itself, you get $XYXY$. Notice that YX (which is $s2$) is perfectly nestled right in the middle of $XYXY$! Therefore, you only need to check if $s1 + s1$ contains $s2$. Time Complexity: $O(n)$ (Assuming the underlying `.contains()` method uses an optimized $O(n)$ substring search). Space Complexity: $O(n)$ to allocate the $s1 + s1$ string in memory.
- 2. The Stream Approach (Functional Offset Matching)** If an interviewer explicitly forbids you from allocating extra space (e.g., you cannot create a concatenated string), you must fall back to checking character by character using modulo math. The Logic: We use an outer `IntStream` to iterate through every possible rotation starting point (offset i). We use an inner `IntStream` to check if every character in $s2$ matches the character in $s1$ offset by i . We use the modulo operator $(i + j) \% n$ to force the index to wrap back around to the front of $s1$ if it goes out of bounds. Time Complexity: $O(n^2)$ because in the worst-case scenario (e.g., checking "aaaaaaaaab" against "aaaaaaaaac"), it has to evaluate almost the entire inner loop for every outer loop iteration. Space Complexity: $O(1)$ because we are only evaluating primitive characters in place and never creating new strings or arrays.

Here are the optimal implementations for finding the longest word, reversing the words, and finding repeating words within a sentence. When dealing with full sentences, a key optimization standard is safely handling punctuation and multiple consecutive whitespaces. The solutions below address these challenges efficiently.

1. LongestWordFinder

The goal is to find the word with the highest character count. To make it precise, punctuation characters (like commas or periods) attached to a word must be stripped out before calculating its length.

Java package com.codes.java.codingquestions;

```
import java.util.Arrays;
```

```
import java.util.Comparator;
```

```
public class LongestWordFinder {
```

```
    // 1. Without Streams (Optimal Loop: O(n) Time, O(w) Space)
```

```
    public static String findLongestWord(String sentence) {
```

```
        if (sentence == null || sentence.trim().isEmpty()) {
            return "";
        }
```

```
        // Split by one or more spaces
```

```
        String[] words = sentence.split("\\s+");
```

```
        String longestWord = "";
```

```
        for (String word : words) {
```

```
            // Clean word from punctuation to get an accurate character length
```

```
            String cleanedWord = word.replaceAll("[^a-zA-Z0-9]", "");
```

```
            if (cleanedWord.length() > longestWord.length()) {
                longestWord = cleanedWord;
            }
        }
```

```
    }
```

```
    return longestWord;
```

```
}
```

```
    // 2. With Streams (Optimal Stream: O(n) Time, O(w) Space)
```

```
    public static String findLongestWordStream(String sentence) {
```

```
        if (sentence == null || sentence.trim().isEmpty()) {
            return "";
        }
```

```
        return Arrays.stream(sentence.split("\\s+"))
```

```
            .map(word -> word.replaceAll("[^a-zA-Z0-9]", "")) // Clean punctuation
```

```
            .max(Comparator.comparingInt(String::length)) // Find maximum length
```

```
            .orElse("");
```

```
}
```

```
// Test Cases
```

```

public static void run() {
    System.out.println("--- Find Longest Word ---");
    String[] tests = {
        "The quick brown fox jumps over the lazy dog.", // "jumps"
        "Welcome to Java programming!",                // "programming"
        "A magnificent, ultra-long sentence.",          // "magnificent"
        " ",                                             // ""
        null                                             // ""
    };

    for (String test : tests) {
        String displayStr = (test == null) ? "null" : "\"" + test + "\"";
        System.out.println("Input: " + displayStr +
            " | Loop: \"" + findLongestWord(test) + "\"" +
            " | Stream: \"" + findLongestWordStream(test) + "\"");
    }
}

```

Explanation of the ApproachesWithout Streams: We split the sentence into a string array using `\s+` (which groups multiple consecutive spaces into a single split boundary). Inside the loop, `word.replaceAll("[^a-zA-Z0-9]", "")` filters out any trailing punctuation like commas or periods so that "programming!" accurately evaluates to a length of 11 instead of 12. With Streams: We stream the split array, perform the regex cleanup inside a `.map()` transformation, and use the terminal `.max()` operator paired with `Comparator.comparingInt(String::length)` to safely extract the longest match.

2. SentenceWordReverserThe goal is to flip the order of the words in the sentence while maintaining the words themselves intact (e.g., "Hello World" becomes "World Hello").

javapackage
com.codes.java.codingquestions;

```

import java.util.Arrays;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

```

```

public class SentenceWordReverser {

    // 1. Without Streams (Optimal Reverse Append: O(n) Time, O(n) Space)
    public static String reverseWords(String sentence) {
        if (sentence == null || sentence.trim().isEmpty()) {
            return sentence;
        }

        String[] words = sentence.trim().split("\\s+");
        StringBuilder sb = new StringBuilder();

        // Iterate backwards through the array of words
        for (int i = words.length - 1; i >= 0; i--) {
            sb.append(words[i]);
            if (i > 0) {
                sb.append(" "); // Add space separator between words
            }
        }
    }
}

```

```

    }
}
return sb.toString();
}

// 2. With Streams (Optimal Inverse IntStream Mapping: O(n) Time, O(n) Space)
public static String reverseWordsStream(String sentence) {
    if (sentence == null || sentence.trim().isEmpty()) {
        return sentence;
    }

    String[] words = sentence.trim().split("\\s+");

    // Map indices backwards to pull words in reverse order
    return IntStream.range(0, words.length)
        .mapToObj(i -> words[words.length - 1 - i])
        .collect(Collectors.joining(" "));
}

// Test Cases
public static void run() {
    System.out.println("\n--- Reverse Words in Sentence ---");
    String[] tests = {
        "Java coding question",
        " Fly me to the moon ",
        "SingleWord",
        "",
        null
    };

    for (String test : tests) {
        String displayStr = (test == null) ? "null" : "\"" + test + "\"";
        System.out.println("Input: " + displayStr);
        System.out.println(" Loop: \"" + reverseWords(test) + "\"");
        System.out.println(" Stream: \"" + reverseWordsStream(test) + "\"");
    }
}
}

```

Explanation of the Approaches Without Streams: We trim leading/trailing spaces and split the sentence. Then, we execute a reverse for loop starting from words.length - 1 down to 0. We build our final string sequence using a StringBuilder to minimize string allocations. With Streams: Java Streams process sequentially forward by default. To reverse via a pipeline without using heavy boxing operations like Collections.reverse(), we feed a forward-running IntStream.range(0, words.length) and mathematically invert the pointer indices inside the mapping phase: words.length - 1 - i. Finally, Collectors.joining(" ") effortlessly stitches them together.

3. RepeatingWordsFinder

The goal here is to find all words that appear more than once inside a given sentence. Word evaluation should generally be case-insensitive (e.g., "The" and "the" are treated as duplicates).

Java package com.codes.java.codingquestions;

```

import java.util.Arrays;
import java.util.Collections;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Map;
import java.util.Set;
import java.util.function.Function;
import java.util.stream.Collectors;

public class RepeatingWordsFinder {

    // 1. Without Streams (Optimal Two-Set Approach: O(n) Time, O(w) Space)
    public static Set<String> findRepeatingWords(String sentence) {
        if (sentence == null || sentence.trim().isEmpty()) {
            return Collections.emptySet();
        }

        String[] words = sentence.toLowerCase().split("\\s+");
        Set<String> seen = new HashSet<>();
        Set<String> duplicates = new LinkedHashSet<>(); // Keeps insertion order of duplicates

        for (String word : words) {
            String cleaned = word.replaceAll("[^a-zA-Z0-9]", "");
            if (!cleaned.isEmpty()) {
                // If it fails to add to 'seen', it is a repeating word
                if (!seen.add(cleaned)) {
                    duplicates.add(cleaned);
                }
            }
        }
        return duplicates;
    }

    // 2. With Streams (Optimal Frequency Map: O(n) Time, O(w) Space)
    public static Set<String> findRepeatingWordsStream(String sentence) {
        if (sentence == null || sentence.trim().isEmpty()) {
            return Collections.emptySet();
        }

        return Arrays.stream(sentence.toLowerCase().split("\\s+"))
            .map(word -> word.replaceAll("[^a-zA-Z0-9]", ""))
            .filter(word -> !word.isEmpty())
            // Group words by their appearance count
            .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
            .entrySet().stream()
            .filter(entry -> entry.getValue() > 1) // Filter out unique words
            .map(Map.Entry::getKey)
            .collect(Collectors.toCollection(LinkedHashSet::new));
    }
}

```

```

}

// Test Cases
public static void run() {
    System.out.println("\n--- Find Repeating Words ---");
    String[] tests = {
        "The track was wet, so the racecars spun out on the track.",
        "Hello world, hello Java!",
        "No duplicate words here",
        "",
        null
    };

    for (String test : tests) {
        String displayStr = (test == null) ? "null" : "\"" + test + "\"";
        System.out.println("Input: " + displayStr +
            " | Loop: " + findRepeatingWords(test) +
            " | Stream: " + findRepeatingWordsStream(test));
    }
}

```

Explanation of the ApproachesWithout Streams: We transform the entire string to lowercase to ensure case-insensitivity. As we loop through the isolated words, we utilize `seen.add(cleaned)`. If a word has been evaluated previously, `.add()` evaluates to false, signaling a duplicate. This duplicate is immediately logged into a tracking `LinkedHashSet` to ensure results are displayed in order of discovery.

With Streams: The stream processes the incoming tokens, strips punctuation, and builds a comprehensive frequency chart via `.collect(Collectors.groupingBy(..., Collectors.counting()))`. A subsequent sub-stream evaluates this map's entries, filters out keys whose values are not greater than 1, and packages the repeating words neatly into an ordered Set container.

While "Add Underscore" can mean a few different things, in the context of Java coding interviews, this almost always refers to converting camelCase or PascalCase strings into snake_case. (Note: If you simply meant adding an underscore between every single character in a word like "hello" -> "h_e_l_l_o", the optimal production way to do that is a one-liner: `String.join("_", str.split(""))`;) Assuming you are tackling the classic CamelCase to snake_case interview question, the goal is to detect uppercase letters, insert an underscore right before them, and convert the entire string to lowercase. Here is the complete class featuring the iterative approach, the stream approach, and the production Regex approach. UnderscoreAdderJavapackage com.codes.java.codingquestions;

```
import java.util.stream.Collectors;
```

```
public class UnderscoreAdder {
```

```
    // 1. Without Streams (Optimal Iteration: O(n) Time, O(n) Space)
```

```
    public static String addUnderscores(String str) {
```

```
        if (str == null || str.isEmpty()) {
```

```
            return str;
```

```
        }
```

```
        StringBuilder sb = new StringBuilder();
```

```
        for (int i = 0; i < str.length(); i++) {
```

```
            char c = str.charAt(i);
```

```
            // When we find an uppercase letter...
```

```
            if (Character.isUpperCase(c)) {
```

```
                // Add an underscore, but ONLY if it's not the very first character
```

```
                if (i > 0 && sb.charAt(sb.length() - 1) != '_') {
```

```
                    sb.append('_');
```

```
                }
```

```
                sb.append(Character.toLowerCase(c));
```

```
            } else {
```

```
                sb.append(c);
```

```
            }
```

```
        }
```

```
        return sb.toString();
```

```
    }
```

```
    // 2. With Streams (Functional Mapping: O(n) Time, O(n) Space)
```

```
    public static String addUnderscoresStream(String str) {
```

```
        if (str == null || str.isEmpty()) {
```

```
            return str;
```

```
        }
```

```
        // Isolate the first character to prevent placing a leading underscore (e.g., "_java_code")
```



```

String firstChar = String.valueOf(Character.toLowerCase(str.charAt(0)));

return firstChar + str.substring(1).chars()
    .mapToObj(c -> (char) c)
    .map(c -> Character.isUpperCase(c) ? "_" + Character.toLowerCase(c) : String.valueOf(c))
    .collect(Collectors.joining());
}

// 3. The Production Approach (Regex)
public static String addUnderscoresRegex(String str) {
    if (str == null || str.isEmpty()) {
        return str;
    }

    // Find boundaries where a lowercase letter is immediately followed by an uppercase letter
    return str.replaceAll("([a-z])([A-Z]+)", "$1_$2").toLowerCase();
}

// Test Cases
public static void run() {
    System.out.println("--- Add Underscores (CamelCase to snake_case) ---");

    String[] tests = {
        "javaCodingQuestion", // Standard camelCase
        "PascalCaseString", // Standard PascalCase (Starts with capital)
        "already_snake_case", // No changes needed
        "a", // Single character
        "",
        null
    };

    for (String test : tests) {
        String displayStr = (test == null) ? "null" : "\"" + test + "\"";
        System.out.println("Input: " + displayStr);
        System.out.println(" Loop: \"\" + addUnderscores(test) + \"\"");
        System.out.println(" Stream: \"\" + addUnderscoresStream(test) + \"\"");
        System.out.println(" Regex: \"\" + addUnderscoresRegex(test) + \"\"");
        System.out.println();
    }
}
}

```

Explanation of the Approaches

Note on Space Complexity: We are generating an entirely new string based on the input, which inherently requires $O(n)$ space complexity.

- Without Streams (The for Loop)** This is the standard algorithmic approach. We loop through the string character by character. We use `Character.isUpperCase(c)` to detect boundaries. The most critical part of this logic is the edge-case check: `i > 0`. If the string is in PascalCase (e.g., "JavaCoding"), the first letter is capitalized. If we don't check `i > 0`, we will accidentally output `"_java_coding"`. We only add the underscore if we are past the first index.
- With Streams (IntStream Mapping)** Streams are not natively great at handling "look-

behind" state (checking if the previous character means we shouldn't add an underscore to the first letter). To make this purely functional, we cheat slightly by handling the `firstChar` manually and converting it to lowercase. We then create a stream from `str.substring(1)` (the rest of the string). Inside the `.map()`, we use a simple ternary operator: if the character is uppercase, transform it into a string with a leading underscore and a lowercase letter. Otherwise, just return the character as a string. `Collectors.joining()` stitches the stream back together.

3. The Production Approach (Regex)

If you were given a ticket to do this in a real enterprise application, Regex is the cleanest way. `[a-z]` is Capture Group 1: It finds a lowercase letter. `[A-Z]+` is Capture Group 2: It finds the uppercase letter that immediately follows it. `"$1_$2"` tells the replacer: "Put Capture Group 1 back, add an underscore, and put Capture Group 2 back." Finally, we chain `.toLowerCase()` at the end to make the whole thing valid `snake_case`.

This EPERM: operation not permitted error happens because Node.js is trying to read or resolve a path inside the C:\Users\Administrator directory, but the current terminal session doesn't have the required permissions to access it. Even though your project is on your D: drive (D:\SAIB Repos\...), npm and Node.js often reach out to the C: drive to check global caches, configurations (.npmrc), or environment variables. Here are the most effective ways to fix this, ordered from easiest to most thorough.

1. Run Your Terminal as Administrator Since the error explicitly mentions the Administrator folder, the fastest fix is usually to grant your terminal the rights it's asking for. If you are using VS Code: Close VS Code entirely. Right-click the VS Code shortcut and select "Run as Administrator". Open your integrated terminal and try `npm run startDev` again. If you are using standalone PowerShell: Right-click the PowerShell icon, select "Run as Administrator", navigate to your D: drive project folder, and run the command.
2. Check Your Global npm Configuration Sometimes, npm is accidentally configured to use the Administrator's user profile for its cache or global installations, which causes permission clashes when you run as a standard user. Run these two commands in your terminal to see where npm is looking:
PowerShell
`npm config get prefix`
`npm config get cache`

If either of these outputs a path pointing to C:\Users\Administrator and you are not logged in as the Administrator, you need to change them to your current user profile. You can reset them by running:
PowerShell
`npm config set prefix "C:\Users\YOUR_USERNAME\AppData\Roaming\npm"`
`npm config set cache "C:\Users\YOUR_USERNAME\AppData\Local\npm-cache"`

(Replace YOUR_USERNAME with your actual Windows username).

3. Clear the Cache and Reinstall Dependencies If a previous npm install was run with elevated privileges, some files inside your node_modules might now be owned by the Administrator account. When your current user tries to read them, it gets blocked. To nuke the local project state and start fresh, run these commands sequentially in your project folder:
PowerShell
1. Delete the node_modules folder and lock file
`Remove-Item -Recurse -Force node_modules`
`Remove-Item -Force package-lock.json`

2. Force clean the npm cache
`npm cache clean --force`

3. Reinstall dependencies
`npm install`

4. Check Antivirus / Windows Defender Occasionally, strict Windows security policies or Antivirus software (like Windows Defender or third-party AVs) will block Node's Istat system call from reading sensitive user directories. If the steps above don't work, temporarily disable your antivirus or add an exclusion for node.exe to see if it allows the script to run.